

Python Virtual Environment

How to Create, Use and Share.

Aaron Nugent

Version 1.0.1



What is a Python virtual environment?

A Python virtual environment is essentially a container for each Python project.

Benefits

- Each container has its own Python interpreter.
- Has its own set of installed packages.
- Can have different versions of the same packages.
- Prevents conflicts between projects with packages.
- Keeps your Python installation clean.
- Allows testing with different Python versions.

Creating a Virtual Environment in:

Interactive Environments

```
module load Python
python -m venv myproject_venv
source myproject_venv/bin/activate
deactivate
```

Batch Jobs

```
#!/bin/bash
#SBATCH --job-name=analysis
#SBATCH --time=01:00:00
#SBATCH --ntasks=1

module load Python
source /project/myproject_venv/bin/activate
python my_script.py
```

Note

Virtual environments paths are not relocatable so be sure to plan the location first.

Sharing Dependencies Practical

- Create and activate a Python virtual environment.
- Install numpy using pip.
- View the scripts in /venv/bin.
- Using the following command and outside your environment directory save your dependencies: `python -m pip freeze > requirements.txt`
- deactivate your current virtual environment and create and activate a new one.
- Using the following command install your old dependencies: `python -m pip install -r requirements.txt`

Conclusion

Python virtual environments are a best practice to manage dependencies and avoid project conflicts, keeping your development environment clean, organised and maintainable.